International Institute of Information Technology, Bhubaneswar

Department of Computer Science and Engineering

Object Oriented Programming Laboratory

Lab 5 – C++ Programming: Function Overloading

Date: 28.08.2026 **Semester:** B.Tech 3rd Semester **Section:** CSE B1

Instructions

- Write all programs in **C++**.
- Use **function overloading** to solve each problem.
- Use the same function name for the overloaded versions.
- Differentiate overloaded functions using the **number, type, or order of parameters**.
- Arrays and pointers should be used as function parameters wherever specified.
- Display the output in a neat and readable format.
- Do not create separate function names for operations that are required to demonstrate function overloading.

1. **Distance Converter**
   Create overloaded functions named `convert()` to perform the following conversions:

   - Convert a distance given in kilometers into meters.
   - Convert a distance given in meters into centimeters.
   - Convert a floating-point distance given in kilometers into meters.

   Demonstrate all overloaded versions from the `main()` function and display the converted values.

   Hint: *Use different parameter types or parameter lists so that the compiler can distinguish the overloaded functions.*

2. **Area Calculator**
   Create overloaded functions named `area()` to calculate:

   - the area of a square,
   - the area of a rectangle,
   - the area of a circle.

   Accept the required dimensions from the user and display the area calculated by each overloaded function.

   Hint: *Use the number of parameters to distinguish the functions for different shapes.*

3. **Character Analyzer**
   Create overloaded functions named `check()` to perform the following operations:

   - Determine whether an integer is positive, negative, or zero.
   - Determine whether a character is an uppercase or lowercase letter.
   - Search for a specified character in a character array.

   Display the result of each operation.

   Hint: *Use different parameter types and parameter lists to create the overloaded versions.*

4. **Array Processing**
   Create overloaded functions named `process()` to perform the following:

   - Calculate the sum of all elements of an integer array.
   - Calculate the sum of all elements of a floating-point array.
   - Calculate the sum of only the first k elements of an integer array.

   Accept the arrays and required values from the user and display the calculated sums.

   Hint: *Use the array and its size as parameters. The third version should contain an additional parameter specifying the number of elements to process.*

5. **Swap Values**
   Create overloaded functions named `swapData()` to swap values in the following cases:

   - Swap two integer values using references.
   - Swap two floating-point values using references.
   - Swap two integer values using pointers.

   Display the values before and after swapping.

   Hint: *Observe how the parameter types differ when references and pointers are used.*

6. **String Information**
   Create overloaded functions named `information()` to perform the following operations:

   - Find the length of a character array.
   - Count the occurrence of a specified character in a character array.
   - Count the occurrence of a specified character within the first k positions of a character array.

   Display the result of each operation.

   Hint: *Use the additional parameter in the third function to specify how many positions should be examined.*

7. **Nearest Value**
   Create overloaded functions named `nearValue()` to determine:

   - Which of two integers is closer to zero.
   - Which of two floating-point values is closer to zero.
   - Which element of an integer array is closest to zero.

   Display the selected value in each case.

   Hint: *For the array version, pass the array along with its size. You may use the absolute value of a number to compare its distance from zero.*

8. **Update Array Elements**
   Create overloaded functions named `update()` to perform the following:

   - Increase an integer variable by a specified amount.
   - Increase a floating-point variable by a specified amount.
   - Increase every element of an integer array by a specified amount.

   Display the values before and after the update.

   Hint: *Use references or pointers when the original variable or array needs to be modified.*

9. **Data Inspection Using Pointers**
   Create overloaded functions named `inspect()` to:

   - Display the value of an integer variable.
   - Display the value stored at an integer pointer.
   - Display all elements of an integer array using a pointer and its size.

   Demonstrate all overloaded functions from `main()`.

   Hint: *Pay attention to the difference between an `int` parameter and an `int*` parameter.*

10. **Result Evaluator**
    Create overloaded functions named `evaluate()` to perform the following operations:

    - Calculate the average of two integers.
    - Calculate the average of three integers.
    - Calculate the average of two floating-point values.
    - Calculate the average of all elements of an integer array.
    - Calculate the average of two integer values accessed through pointers.

    Accept the required inputs and display the result produced by each overloaded function.

    Hint: *Carefully design the parameter lists so that each version is uniquely identified by the compiler.*